

Wireshark packet capture

1. Purpose

The purpose of this task was to conduct a network traffic analysis using **Wireshark** to examine HTTP communication between a web browser and an **OWASP Juice Shop** application. The objective was to determine how HTTP traffic generated during an XSS testing exercise could be observed at the network level.

The analysis focused on identifying HTTP requests and responses associated with the Juice Shop application and determining whether an XSS payload was visible within the captured network traffic.

2. Objective

- To capture HTTP traffic generated when accessing OWASP Juice Shop.
- To identify the communication between the browser and the Juice Shop server.
- To inspect HTTP requests and responses using Wireshark.
- To determine whether an XSS payload could be observed in the captured traffic.
- To use Wireshark's HTTP stream functionality to inspect the complete communication.
- To provide network-level evidence that can support an XSS security testing report.

3. Methodology

A display filter was used to isolate traffic involving the Juice Shop server and port 3000:

```
ip.addr == 192.168.18.81 && tcp.port == 3000
```

This reduced unrelated network traffic and made it easier to identify HTTP communication between the browser and Juice Shop.

The browser was then used to interact with the Juice Shop application while Wireshark captured the resulting packets.

The captured HTTP traffic was inspected by selecting individual packets and examining the HTTP protocol information. The **Follow** → **HTTP Stream** functionality was also used to reconstruct the communication between the client and server.

4. Notes taken

XSS payload:

```
<img src=x onerror=alert(XSS)>
```

The browser may interpret x as the image source and subsequently attempt to request that resource.

Consequently, Wireshark may capture:

GET /x HTTP/1.1

As a separate browser-generated request. This means that the /x request should not automatically be interpreted as proof that the XSS payload itself was transmitted in that packet.

The actual XSS payload would need to be located in the HTTP request containing the relevant parameter, request body, URI, or other HTTP field.

5. Conclusion

The Wireshark investigation successfully demonstrated the capture and inspection of HTTP traffic generated by an OWASP Juice Shop application running on TCP port 3000.

The captured traffic was examined for common XSS-related strings, including:

- <script
- <img
- onerror
- javascript:
- alert(

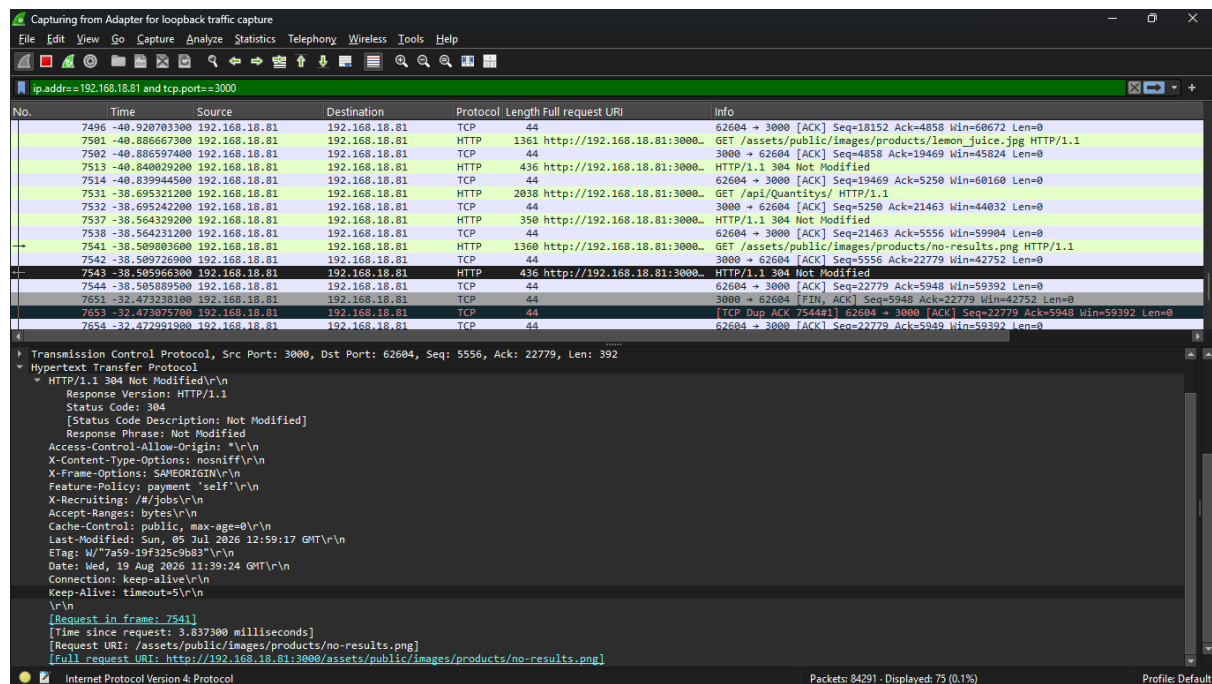
These strings can help determine whether an XSS test payload was transmitted over an unencrypted HTTP connection.

For example, if a payload were transmitted directly in a GET parameter, Wireshark could potentially display something similar to:

GET /search?q=<script>alert(1)</script> HTTP/1.1

WIRESHARK XSS HTTP PACKET CAPTURE SCREENSHOT:

Filter used: ip.addr==192.168.18.81 and port==3000



Payload of XSS using Follow -> HTTP STREAM:

